Cost-Based Query Optimizer

A Standalone Database Optimization Engine

DBMSTeam 404

Roll Number	Name
23CS10054	Popuri Anirudh (c)
23CS10013	Chintada Yesheeth Sree Narayana
23CS10078	Vemuri Sai Lokesh
23CS10052	Payidiparthy Sri Surya Naga Hadwik (vc)
23CS10017	Dasari Veera Venkata Abhinav Teja

Department of Computer Science and Engineering

Course: Database Management Systems Lab
April 13, 2026

kgplogo.png

Contents

Objective

Database query performance is a critical bottleneck in data-intensive applications. As query complexity grows—particularly with multi-table joins—the number of possible execution paths grows exponentially. The goal of this project is to design and implement a **standalone, cost-based query optimizer** that mathematically evaluates and restructures database query plans.

The system accepts raw SQL DML commands, parses them into a logical execution tree, validates the plan against a catalog of table statistics, and produces an optimized physical execution plan using a combination of rule-based heuristics and dynamic-programming-based cost analysis.

Specifically, the project targets the following outcomes:

- A functional SQL parser (Flex/Bison) that converts SQL strings into a structured JSON Abstract Syntax Tree (AST).
- A logical plan representation supporting the core SPJ (Select-Project-Join) operations plus `GROUP BY`, `HAVING`, `ORDER BY`, and `LIMIT`.
- A mock and live catalog system (MockCatalog / PostgresCatalog) for supplying table statistics.
- A cost model implementing Sequential Scan, Index Scan, Nested Loop Join, and Hash Join formulas using CPU, memory, and disk I/O components.
- A semantic analyzer that validates table and column references before any optimization is attempted.
- A Rule-Based Optimizer (RBO) for predicate pushdown.
- A Cost-Based Optimizer (CBO) using bitmask dynamic programming to explore both left-deep and bushy join trees, with early branch pruning.

Methodology

The optimizer was built incrementally over five weeks, with each week targeting a specific layer of the system. The full pipeline is shown in Figure 1.

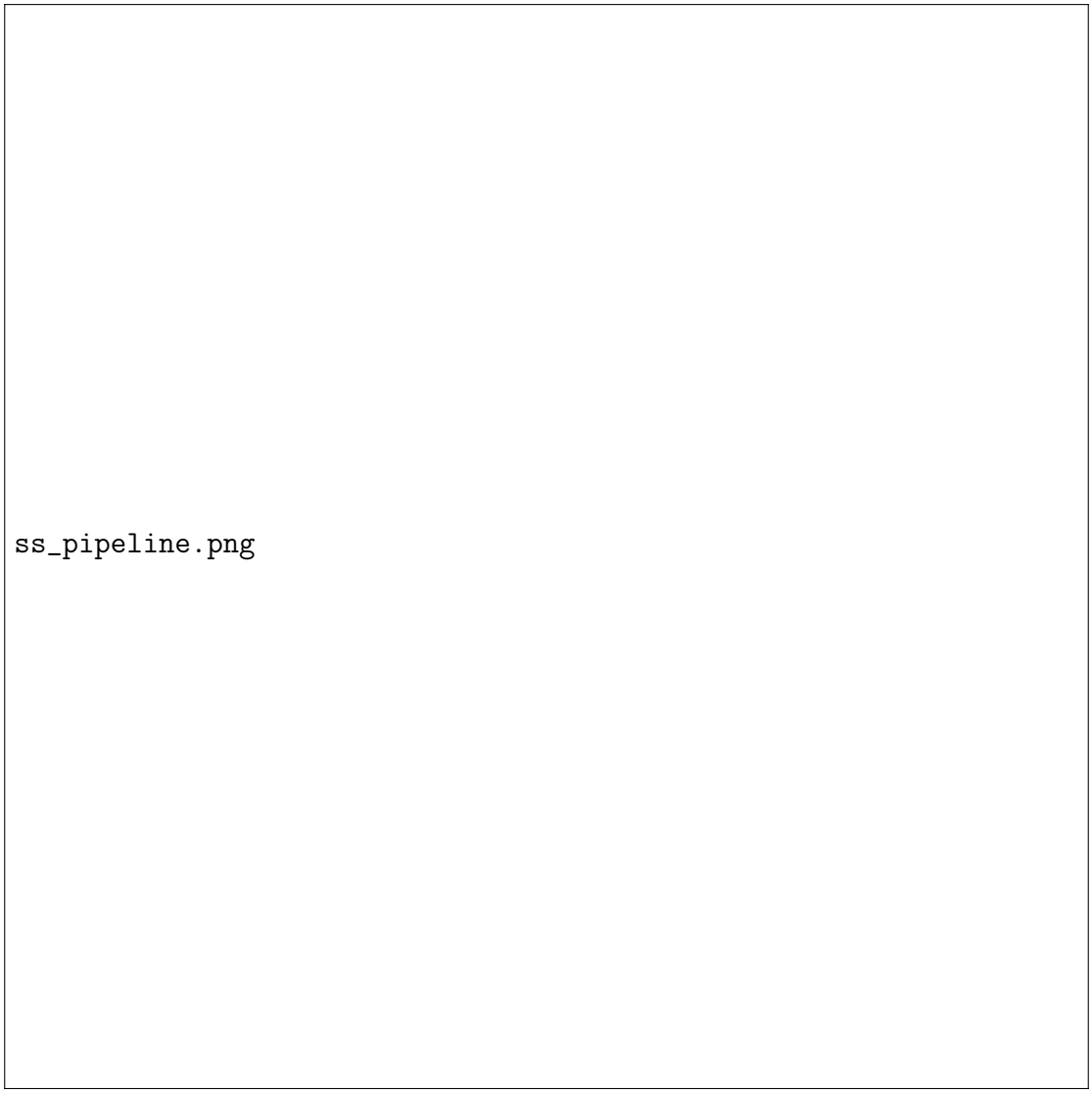

Figure 1: End-to-end optimizer pipeline: SQL input to physical execution plan.

2.1 SQL Parser (Flex / Bison)

The front-end of the optimizer is a Flex lexer and Bison parser that translates raw SQL strings into a JSON AST conforming to a strict API contract agreed upon before implementation began.

The lexer (`lexer.l`) tokenises all SQL keywords, identifiers, numeric and string literals, and comparison operators. The parser (`parser.y`) defines a grammar covering:

- `SELECT` with bare identifiers, qualified names (`table.col`), and `*`.
- `FROM` with optional `AS` alias and bare alias.
- `INNER / LEFT / RIGHT / CROSS JOIN ...ON`.
- `WHERE` and `HAVING` with compound `AND` / `OR` predicates and comparison operators (`=`, `<>`, `<`, `>`, `<=`, `>=`).
- `GROUP BY` (optionally followed by `HAVING`).

Operator precedence is declared explicitly to resolve ambiguity in compound predicates. The grammar outputs a `nlohmann::json` object which is the "API contract" consumed by the C++ optimizer pipeline.

The Bison output is bridged to C++ via a single exported function:

```
nlohmann::json parse_sql_to_json();  // defined in parser.y, called
    from main.cpp
```

Listing 1: Bison-to-C++ bridge

2.2 Logical Plan Representation (AST Nodes)

The JSON AST is deserialised into an in-memory tree of `PlanNode` objects. Seven logical node types are supported, each a subclass of the abstract base:

```
class PlanNode {
public:
    NodeType type;
    std::vector<std::unique_ptr<PlanNode>> children;
    virtual void print(int indent = 0) const = 0;
};
```

Listing 2: PlanNode base class (ast_nodes.hpp)

Table 1: Supported logical node types

SQL Clause	C++ Class	Key Data Stored
FROM	LogicalGetNode	table name, alias
WHERE / HAVING	LogicalFilterNode	predicate expression tree
SELECT	LogicalProjectNode	column list
JOIN ...ON	LogicalJoinNode	join type, ON condition
GROUP BY	LogicalAggregateNode	group columns, aggregates
ORDER BY	LogicalSortNode	sort columns, directions
LIMIT	LogicalLimitNode	row count

Condition expressions (WHERE / ON clauses) are represented as recursive `Expression` structs with five types: `COLUMN`, `CONSTANT`, `COMPARISON`, `LOGICAL_AND`, `LOGICAL_OR`.

2.3 System Catalog

The catalog is defined behind a polymorphic interface:

```
class ICatalog {
public:
    virtual TableStats getTableStats(const std::string& table_name)
        const = 0;
    virtual bool columnExists(const std::string& table_name,
                              const std::string& col_name)   const = 0;
};
```

Listing 3: ICatalog interface (catalog.hpp)

Two concrete implementations were built:

MockCatalog. Reads table statistics and column lists from a JSON configuration file (`src/catalog.json`) at startup. If the file is absent or malformed it falls back to hardcoded defaults. A helper `addTable()` method allows tables to be registered programmatically at runtime, useful for unit testing.

PostgresCatalog. Uses `popen()` to invoke `psql` in a hidden subprocess, running `ANALYZE` and then querying `pg_class` to retrieve real `reltuples` and `relpages` values from a live PostgreSQL instance.

2.4 Cost Model

All cost formulas are implemented as static methods in `CostModel` (cost_model.hpp). The following constants are used throughout:

Table 2: Cost model constants

Constant	Value	Meaning
`SEQ_PAGE_COST`	1.0	Sequential disk block read
`RANDOM_PAGE_COST`	4.0	Random disk seek + read
`CPU_TUPLE_COST`	0.01	Per-tuple CPU processing
`CPU_OPERATOR_COST`	0.0025	Per-comparison CPU cost
`MEM_WRITE_COST`	0.005	Per-tuple hash table write

Sequential Scan.

$$C_{\text{seq}} = B \cdot C_{\text{seq\_page}} + N \cdot C_{\text{cpu\_tuple}}$$

where B = number of blocks, N = number of tuples.

Index Scan (B-Tree, height $h = 3$).

$$C_{\text{idx}} = (h + 1) \cdot C_{\text{rand\_page}} + C_{\text{cpu\_tuple}}$$

Nested Loop Join.

$$C_{\text{NLJ}} = B_{\text{out}} \cdot C_{\text{seq\_page}} + N_{\text{out}} \cdot B_{\text{in}} \cdot C_{\text{seq\_page}} + N_{\text{out}} \cdot N_{\text{in}} \cdot C_{\text{cpu\_op}}$$

Hash Join.

$$C_{\text{HJ}} = (B_{\text{out}} + B_{\text{in}}) \cdot C_{\text{seq\_page}} + N_{\text{in}} \cdot C_{\text{mem\_write}} + (N_{\text{out}} + N_{\text{in}}) \cdot C_{\text{cpu\_tuple}} + N_{\text{out}} \cdot C_{\text{cpu\_op}}$$

The optimizer always places the smaller table as the inner (build) side of a Hash Join, and as the outer (loop driver) of a Nested Loop Join.

2.5 Semantic Analyzer

Before any optimization is attempted, the `SemanticAnalyzer` walks the logical tree bottom-up and validates:

1. Every table referenced in a `FROM` or `JOIN` exists in the catalog (checked via `getTableStats`).
2. Every column referenced in `SELECT`, `WHERE`, or `ON` exists in at least one of the active tables (checked via `columnExists`).
3. Fully qualified column names (`table.col`) have both a valid table prefix in scope and a valid column within that table.

Any violation throws a `std::runtime_error` which is caught and reported as a "Semantic Error", stopping execution before optimization begins.

2.6 Rule-Based Optimizer (Predicate Pushdown)

The `RuleBasedOptimizer` (optimizer.hpp) applies **filter pushdown**: moving `LOGICAL_FILTER` nodes as far down the tree as possible so that rows are eliminated before expensive join operations.

The algorithm works bottom-up. When a `LOGICAL_FILTER` sits directly above a `LOGICAL_JOIN`, it asks the catalog which table owns the filtered column. It then moves the filter node below the join, onto the appropriate child subtree.

A key enhancement handles `AND` predicates: the compound predicate is split into two independent filter nodes, each pushed to the table that owns its column. This allows filters on different tables to be pushed simultaneously:

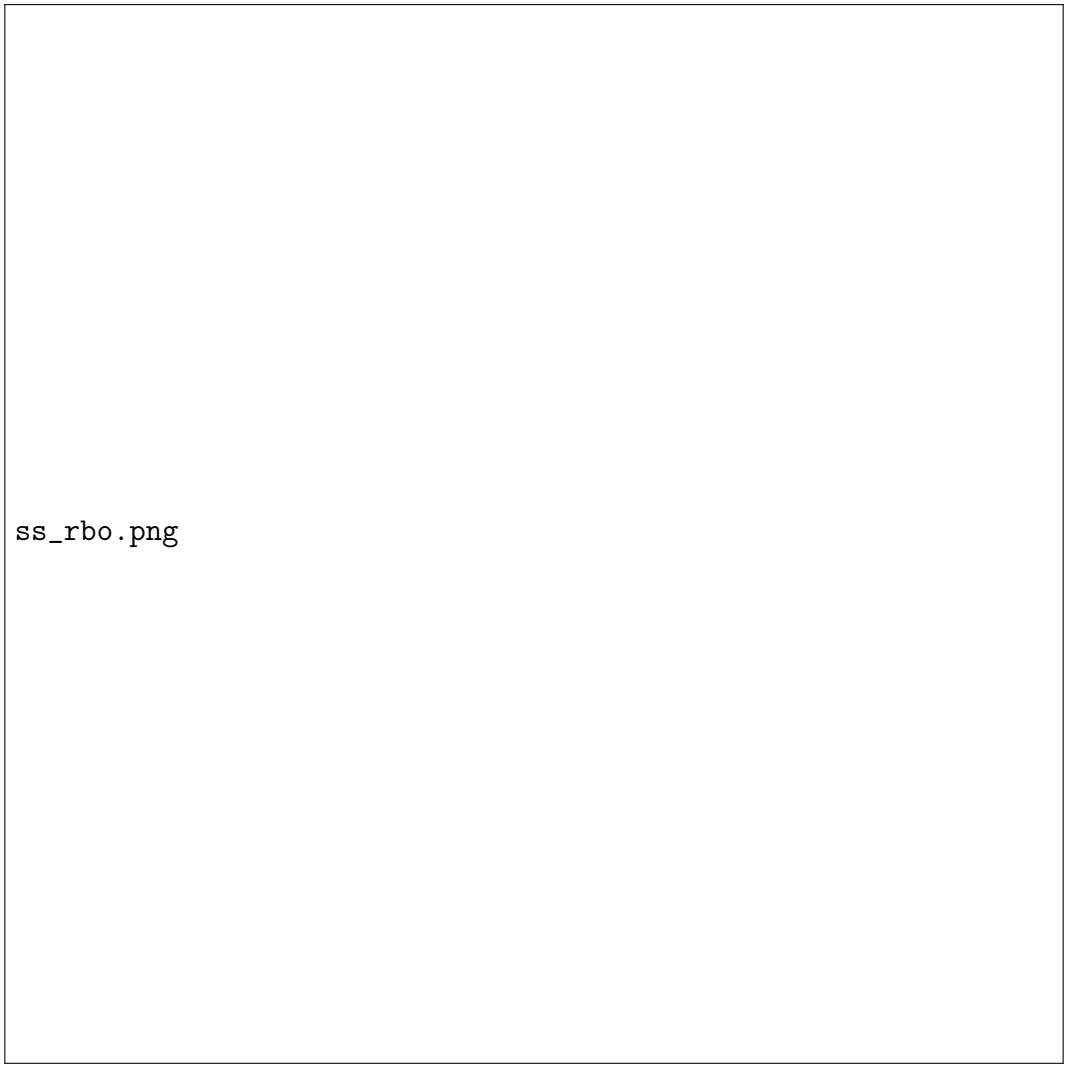

Figure 2: RBO predicate pushdown: `age > 18 AND amount > 500` split and pushed onto `users` and `orders` respectively.

2.7 Cost-Based Optimizer (DP Join Enumeration)

The `CostBasedOptimizer` (cost_based_optimizer.hpp) is the core of the project. It replaces the greedy one-join-at-a-time approach with a **bitmask dynamic programming** algorithm that explores all possible join orderings across both left-deep and bushy tree shapes.

Algorithm overview.

1. **Extract.** Walk the logical tree and collect two lists: `tables[]` (leaf nodes, each with stats and base plan) and `edges[]` (join conditions with their left/right table indices).
2. **Base case.** For each table i, store a `DPEntry` in `memo[`2^i`]` with cost = sequential scan cost and plan = the leaf node (which may be a `FILTER → GET` pair if the RBO already pushed a filter there).
3. **Fill subsets bottom-up.** For every subset size k from 2 to N, iterate all bitmasks S with k bits set (using Gosper's hack for efficient enumeration). For each S, try every

way to split $S = L \cup R$ where $L, R \neq \emptyset$:

$$\text{cost}(S) = \min_{L \cup R = S} \big[\text{cost}(L) + \text{cost}(R) + C_{\text{join}}(L, R)\big]$$

4. **Early pruning.** If $\text{cost}(L) + \text{cost}(R)$ already exceeds the budget threshold (10^7), that split is abandoned immediately without evaluating the join cost.
5. **Cardinality estimation.** The output size of each intermediate join is estimated as:

$$N_{\text{out}} = \max(1,\ N_L \cdot N_R \cdot \sigma)\,, \quad \sigma = 0.1$$

   This intermediate cardinality feeds into the cost of subsequent joins.
6. **Physical algorithm selection.** For each candidate join, both Hash Join and Nested Loop Join costs are computed. The cheaper one is selected and the physical node (`PhysicalHashJoinNode` or `PhysicalNestedLoopJoinNode`) is stored in the memo entry.
7. **Reconstruct.** The best plan for the full bitmask ($2^N - 1$) is extracted from the memo and wrapper nodes (`PROJECT`, `AGGREGATE`, etc.) are re-attached in their original order.

Left-deep vs. bushy trees. Because every way to split a subset S is tried, both left-deep splits (one side is always a single table) and bushy splits (both sides are multi-table sub-joins) are naturally explored in the same loop. No special casing is required.

Complexity. For N tables the DP visits $O(3^N)$ sub-problems (each of the 2^N subsets is split every possible way), which is exponential but acceptable for $N \leq 15$. Early pruning further reduces the practical work for skewed cost landscapes.

Results and Screenshots

All tests were run on WSL (Ubuntu 24 on Windows) using the compiled binary `optimizer_test` against SQL files in the `test_queries/` directory. The MockCatalog was loaded from `src/catalog.json`.

3.1 Test 1: Simple Single-Table Scan

```
SELECT * FROM users;
```

Listing 4: test1_simple_scan.sql

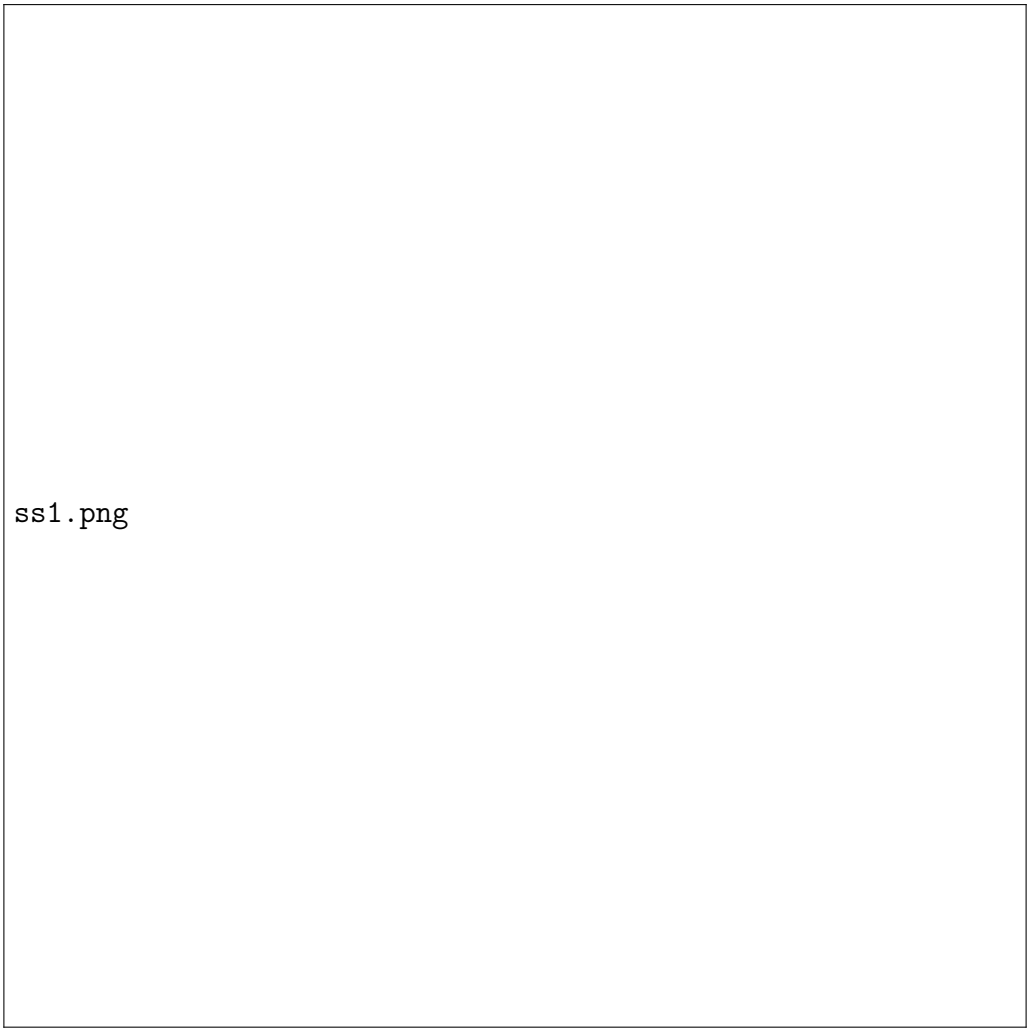

Figure 3: Test 1 output: single-table query — PROJECT wraps GET, no join enumeration.

The optimizer correctly identifies a single-table query and skips join enumeration entirely, producing `LOGICAL_PROJECT → LOGICAL_GET`.

3.2 Test 2: Filter Pushdown (RBO)

```
SELECT name, amount
FROM users
JOIN orders ON users.id = orders.user_id
WHERE age > 18;
```

Listing 5: test2_filter_pushdown.sql

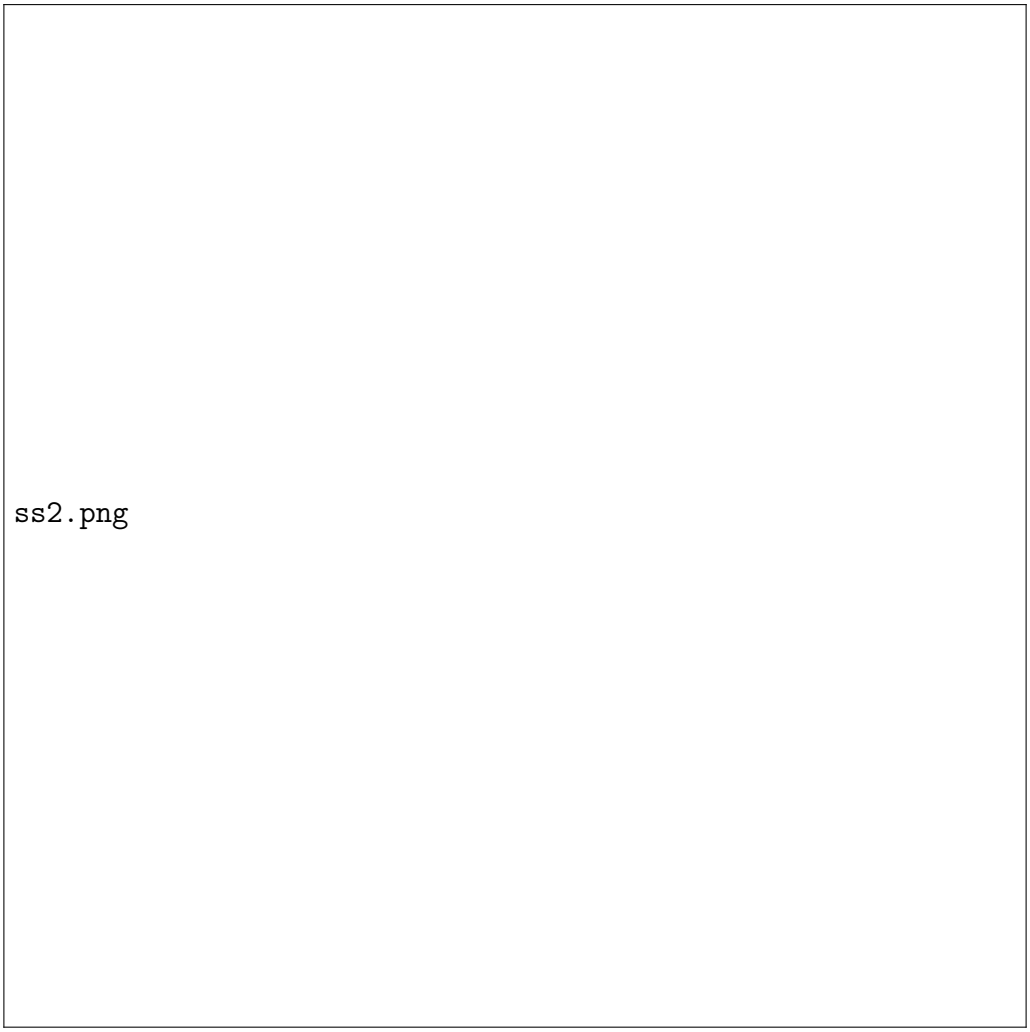

Figure 4: Test 2 output: RBO pushes `FILTER(age > 18)` below the JOIN onto `GET(users)`. CBO selects Hash Join.

The RBO correctly identifies that `age` belongs to `users` and moves the filter node below the join. The filter is preserved in the final CBO output, which selects a Hash Join (users=10k tuples, orders=5M tuples).

3.3 Test 3: Three-Table DP Join Enumeration

```
SELECT name, role_name, amount
FROM users
JOIN orders ON users.id = orders.user_id
JOIN roles  ON users.role_id = roles.id;
```

Listing 6: test3_three_table_join.sql

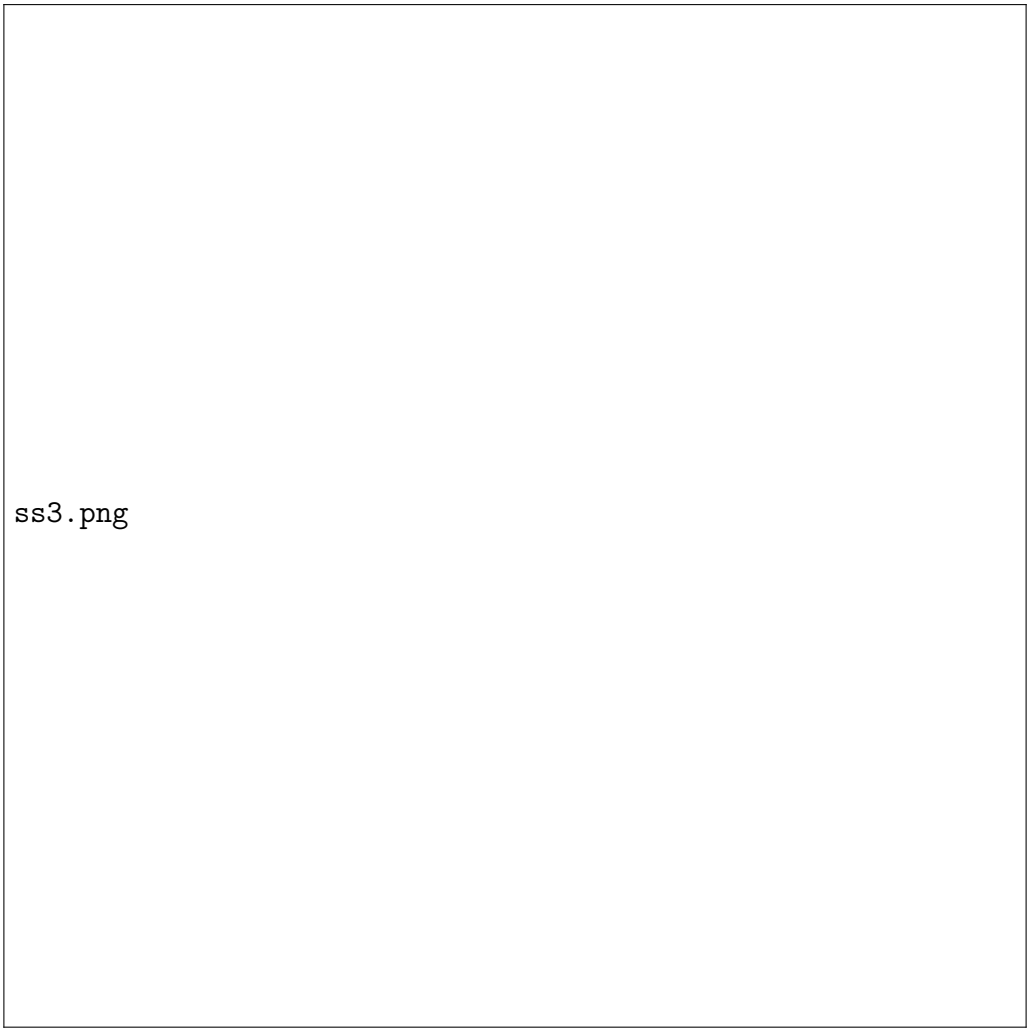

Figure 5: Test 3: DP explores 7 subsets for 3 tables and finds the optimal join order.

This test demonstrates the DP enumerator. The parser produces the naive order `((users JOIN orders) JOIN roles)`, but the DP finds that joining `roles` (50 tuples) with `users` (10,000 tuples) first—cost 428—is far cheaper than joining `users` with `orders` (5,000,000 tuples) first—cost 162,950. The final plan is:

$$\text{orders} \bowtie (\text{users} \bowtie \text{roles}) \quad \text{total cost: } 164{,}178$$

Table 3: DP subset costs for Test 3

Subset bitmask	**Tables**	**Best cost**
`0b00000011`	users + orders	162,950
`0b00000101`	users + roles	428
`0b00000110`	orders + roles	162,503
`0b00000111`	users + orders + roles	164,178

3.4 Test 4: GROUP BY with AGGREGATE Preservation

```sql
SELECT user_id
FROM users
JOIN orders ON users.id = orders.user_id
GROUP BY user_id HAVING amount > 100;
```

Listing 7: test4_groupby_having.sql

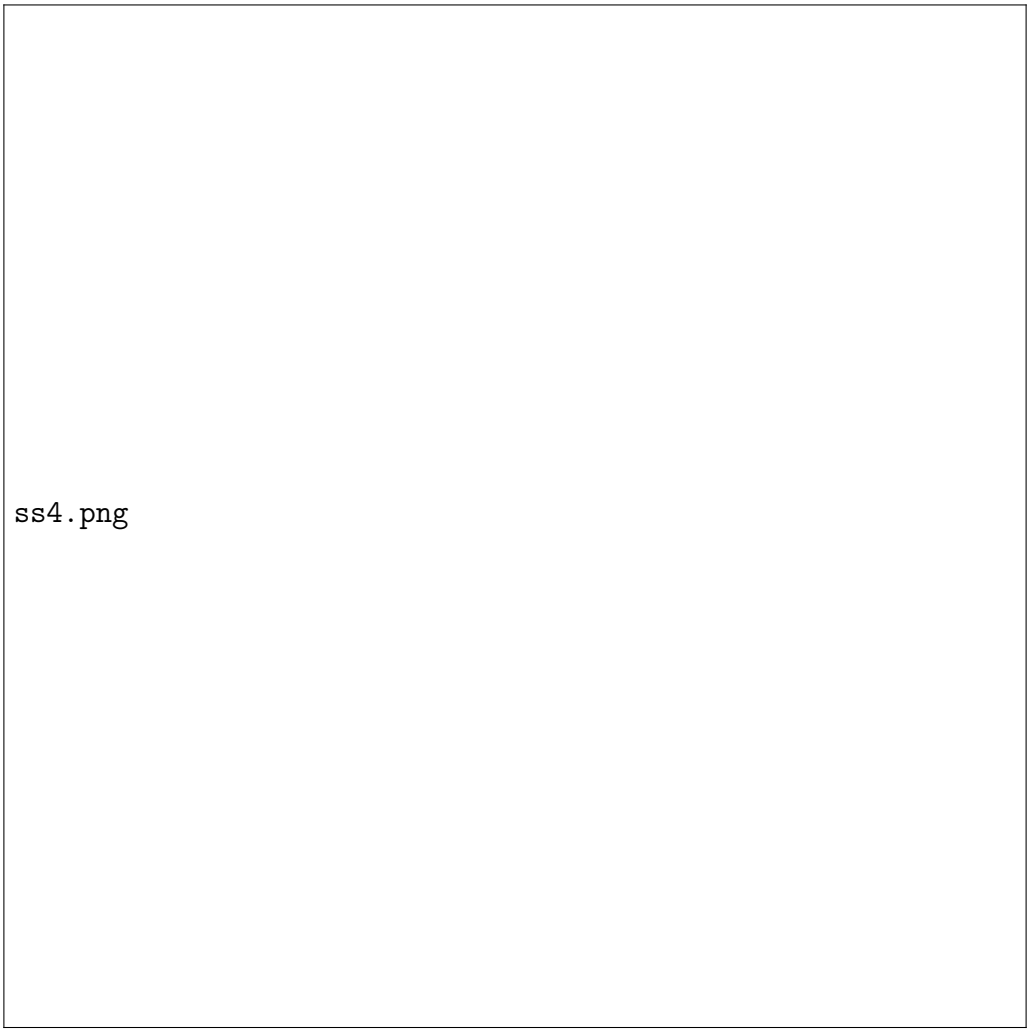

Figure 6: Test 4: `LOGICAL_AGGREGATE` node is correctly re-wrapped around the optimized join core in the final plan.

3.5 Test 5: LEFT JOIN Type Preservation

```sql
SELECT name, amount
FROM users LEFT JOIN orders ON users.id = orders.user_id;
```

Listing 8: test5_left_join.sql

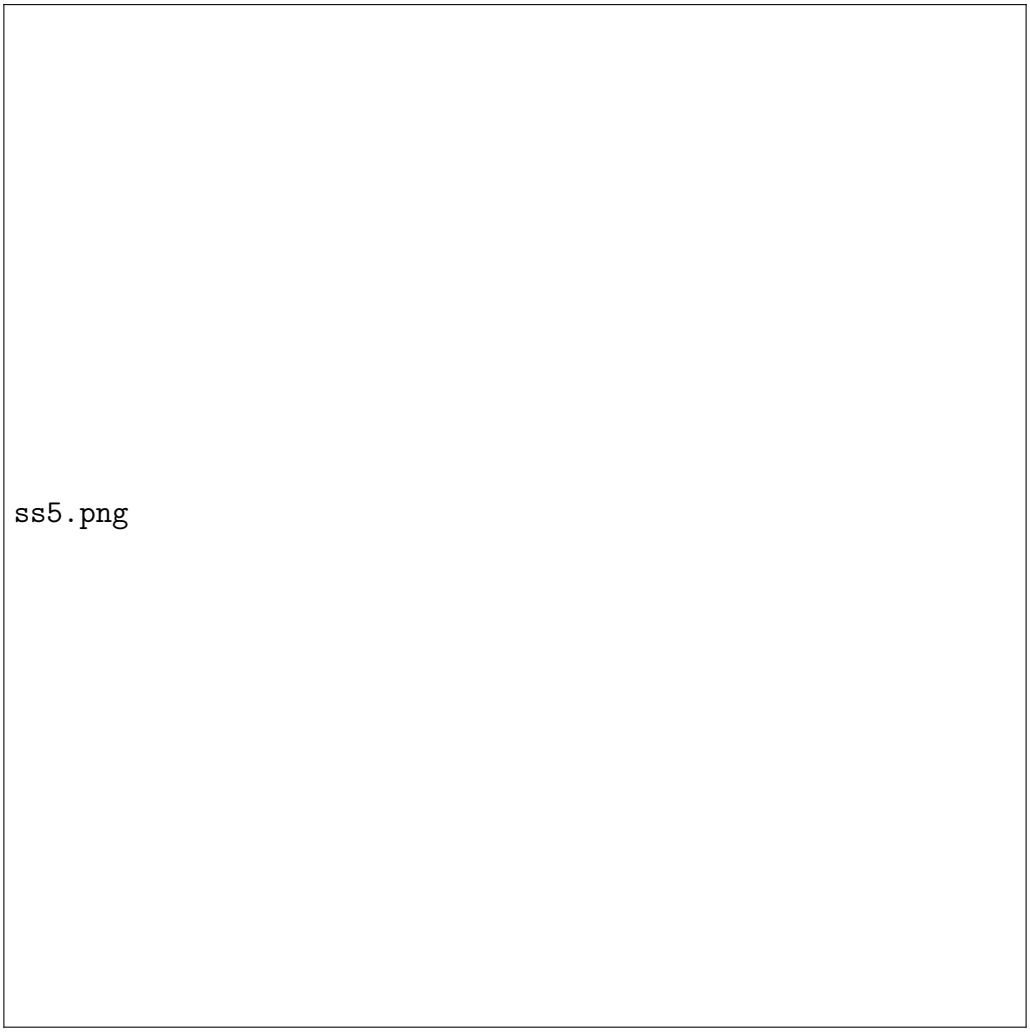

Figure 7: Test 5: `LEFT` join type correctly parsed and preserved through the entire pipeline into the physical node.

3.6 Tests 6 & 7: Semantic Error Detection

```
SELECT * FROM invoices;          -- Table does not exist
SELECT salary FROM users JOIN orders ON users.id = orders.user_id;  --
    Column DNE
```

Listing 9: test6 and test7: semantic errors

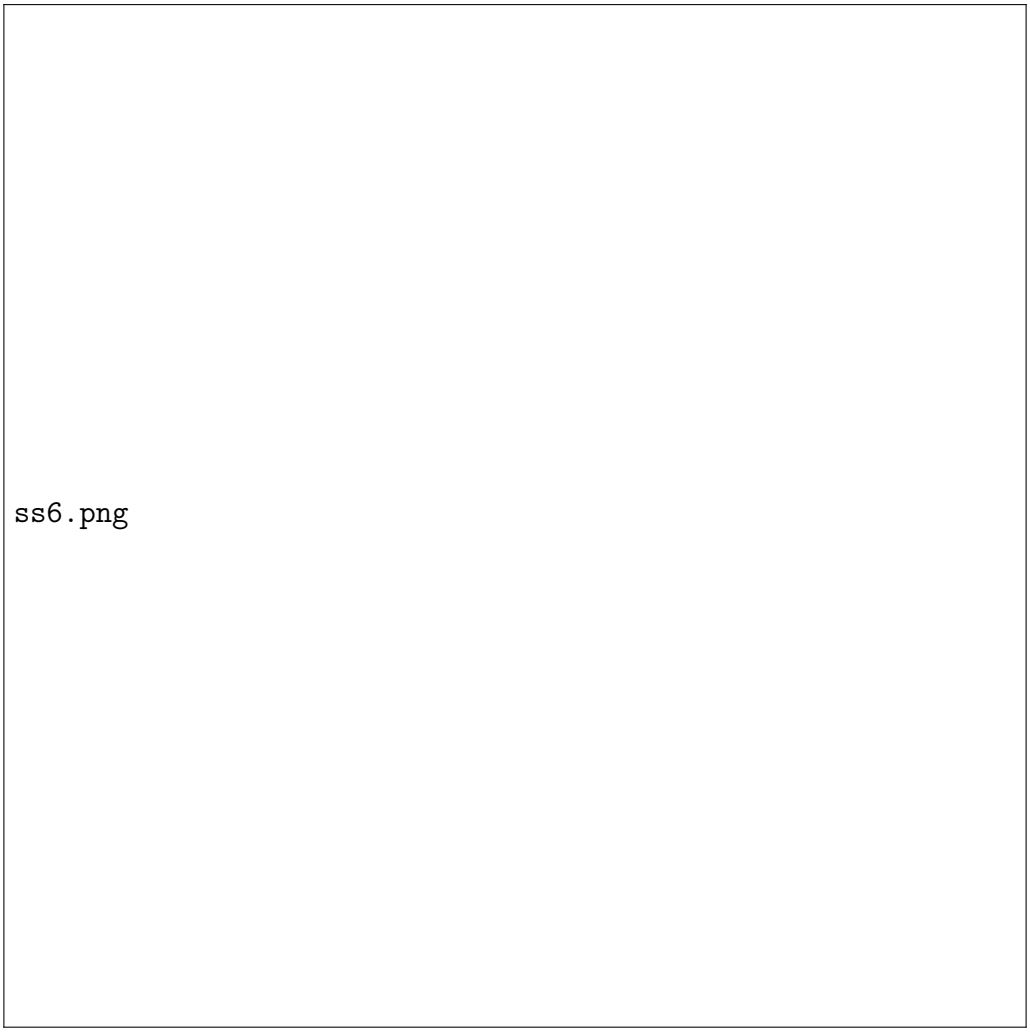

Figure 8: Test 6 and 7: Semantic analyzer correctly catches non-existent table `invoices` and non-existent column `salary`, stopping execution before the optimizer runs.

3.7 Test 8: AND Predicate Split and Dual Pushdown

```
SELECT name, amount
FROM users
JOIN orders ON users.id = orders.user_id
WHERE age > 18 AND amount > 500;
```

Listing 10: test8_and_filter.sql

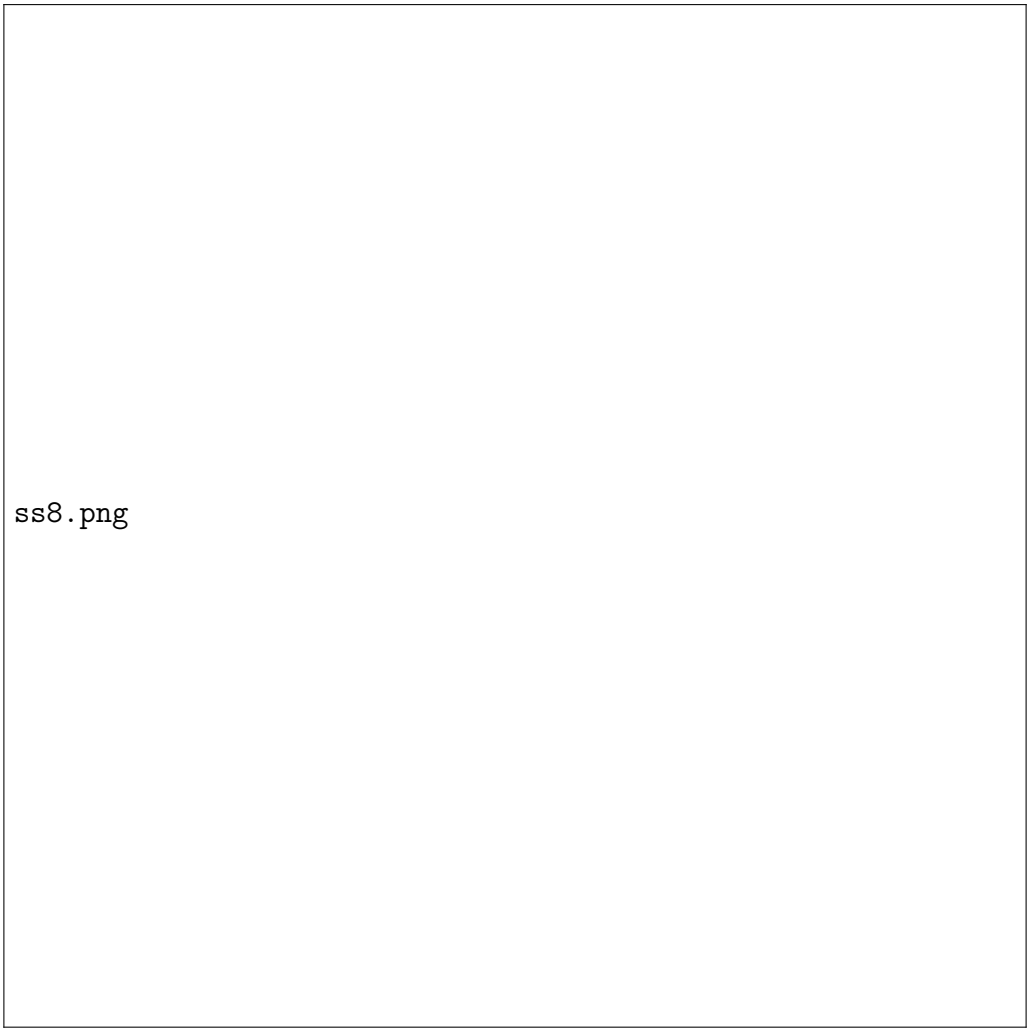

Figure 9: Test 8: `AND` predicate split — `age > 18` pushed onto `users`, `amount > 500` pushed onto `orders`. Both filters preserved in the final plan.

3.8 Summary of Results

Table 4: Test suite results

Test	Description	Key Feature Verified	Result
1	Simple scan	Parser, single table plan	**PASS**
2	Filter pushdown	RBO, filter preserved in CBO	**PASS**
3	3-table DP join	DP enumeration, optimal ordering	**PASS**
4	GROUP BY	AGGREGATE wrapper preservation	**PASS**
5	LEFT JOIN	Join type propagation	**PASS**
6	Bad table	Semantic error — table not found	**PASS**
7	Bad column	Semantic error — column not found	**PASS**
8	AND filter split	Dual predicate pushdown	**PASS**

References

1. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill. [Cost model formulas, join algorithms, and query optimization theory.]
2. Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., & Price, T. G. (1979). Access path selection in a relational database management system. *Proceedings of ACM SIGMOD*, 23–34. [Foundation of cost-based optimization and the System R DP algorithm.]
3. Lohman, G. M. (1988). Grammar-like functional rules for representing query optimization alternatives. *Proceedings of ACM SIGMOD*, 18–27.
4. Graefe, G. (1993). Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2), 73–169.
5. Ioannidis, Y. E., & Kang, Y. C. (1991). Left-deep vs. bushy trees: An analysis of strategy spaces and its implications for query optimization. *Proceedings of ACM SIGMOD*, 168–177.
6. nlohmann/json (2023). JSON for Modern C++. https://github.com/nlohmann/json. [JSON serialization library used for the AST contract.]
7. Flex — The Fast Lexical Analyzer. https://github.com/westes/flex.
8. GNU Bison — The YACC-compatible Parser Generator. https://www.gnu.org/software/bison/.
9. PostgreSQL Documentation: System Catalog `pg_class`. https://www.postgresql.org/docs/current/catalog-pg-class.html.